Teoría del Observador Suficiente

en GraphHunter

Cómo un cazador de amenazas sabe cuándo *puede confiar* en lo que ve

Guía didáctica, paso a paso

Documentación de arquitectura — GraphHunter

23 de julio de 2026

Resumen

Este documento explica, de forma didáctica y paso a paso, cómo la *Teoría del Observador Suficiente* (SOT, por *Sufficient Observer Theory*) organiza la ingesta y la caza de amenazas en GraphHunter. La idea central es simple pero poderosa: antes de responder "¿hay un ataque?", un observador honesto debe poder responder "¿tengo suficiente información para que mi respuesta sea confiable, o hay algo que no puedo ver?". Recorreremos qué es un observador, qué significa que sea *suficiente*, qué es un *testigo de ceguera*, cómo está construida la tubería de datos de GraphHunter, y seguiremos un evento real desde la fuente hasta la respuesta de una hipótesis de caza.

Índice

1. El problema: ver no es lo mismo que saber **2**

2. Vocabulario SOT en cinco ideas **2**
2.1. El testigo de ceguera, el corazón de SOT 2

3. La arquitectura: una tubería que preserva la evidencia **3**

4. Paso a paso: la vida de un evento **4**

5. Los canales: en qué dimensiones puede fallar el observador **5**

6. La auditoría: cómo se *mide* la suficiencia **5**

7. Dos cierres de canal, en detalle **6**
7.1. `ingestion_time`: de "ciego" a "full" 6
7.2. `provenance_stability`: el testigo que casi se nos escapa 6

8. El certificado incremental = rebuild **6**
8.1. Validación por traducción: quién es la base de confianza 6
8.2. Qué compara el comparador (y qué honestamente *no*) 7
8.3. El certificado sobre un dataset *real* 7
8.4. Un límite declarado: `rebuild_faithful` 8

9. El estado final: todo Suficiente, con límites honestos **8**

1. El problema: ver no es lo mismo que saber

Imaginá un analista de seguridad frente a una consola. Escribe una hipótesis de caza — por ejemplo, *"¿algún usuario se autenticó en un host y luego ese host se comunicó con una IP externa sospechosa?"* — y el sistema responde: **"0 resultados"**.

La pregunta incómoda

¿"0 resultados" significa *"no pasó"*, o significa *"pasó, pero no lo vi"*? Estas dos situaciones son operacionalmente idénticas para un sistema ingenuo — y peligrosamente distintas para el analista.

Un firewall pudo haberse caído durante la ventana. Un log pudo llegar tarde y quedar fuera del cursor de ingesta. Una tabla de Sentinel pudo dar timeout. En todos esos casos, un sistema que solo dice "0 resultados" está **mintiendo por omisión**: presenta datos incompletos como si fueran exhaustivos.

SOT ataca exactamente este problema. No busca que el sistema *nunca* sea ciego — eso es imposible en el mundo real — sino que el sistema **sepa cuándo es ciego** y lo **declare honestamente** en su respuesta.

2. Vocabulario SOT en cinco ideas

SOT modela un sistema de observación con cinco elementos. Los presentamos con su nombre formal y su traducción al mundo de GraphHunter.

Símbolo	Nombre	En GraphHunter
X	Mundo real (estados)	Lo que *realmente* pasó en la red: logins, conexiones, procesos.
S	Señal / observación	Lo que las fuentes (Sentinel, Sysmon, FortiGate…) *reportan*.
B	Familia de comportamientos	El catálogo de hipótesis de caza (30 patrones ATT&CK) que queremos poder distinguir.
C	Contextos	Condiciones bajo las que observamos: llegada tardía, caída de fuente, edición de mapping…
R	Respuesta del observador	La respuesta del `run_hunt`: los caminos encontrados *más* los metadatos de cobertura.

Definición de suficiencia

Un observador es **suficiente** para una familia B de comportamientos si, a partir de lo que observa (S), puede **distinguir** cuál comportamiento de B ocurrió — *en todos los contextos C declarados*. Si existe algún contexto donde dos comportamientos distintos producen la *misma observación*, el observador es **insuficiente**, y esa coincidencia se llama un **testigo de ceguera**.

2.1 El testigo de ceguera, el corazón de SOT

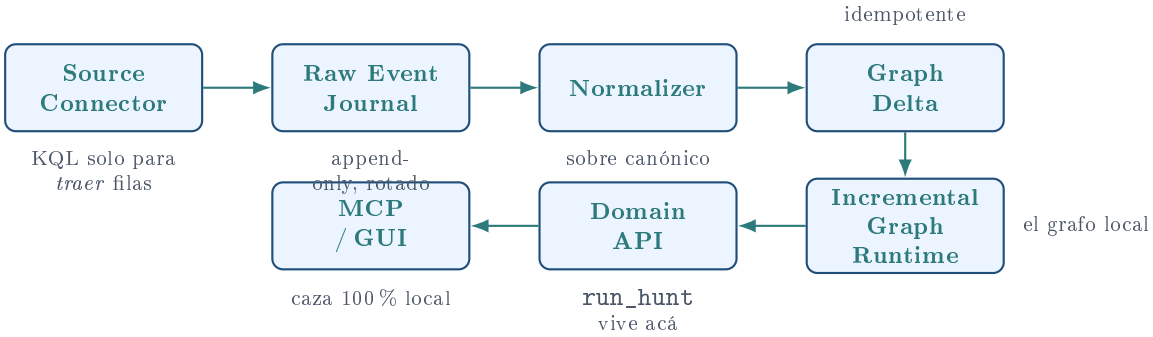
Un **testigo** (witness) es una prueba concreta de que el observador no puede distinguir dos historias que *deberían* distinguirse. Se describe con tres piezas:



Cerrar un testigo significa hacer que `same_observed` **deje de ser verdad**: cambiar el sistema para que las dos historias produzcan respuestas distinguibles. Como veremos, GraphHunter cierra cada testigo con una pieza concreta de código y lo *prueba* con un test.

3. La arquitectura: una tubería que preserva la evidencia

El principio de diseño de GraphHunter es **"KQL no computa el grafo"**: las fuentes (como Microsoft Sentinel) solo entregan *eventos crudos*; toda la correlación y la caza ocurren localmente, en un grafo de procedencia. Esto desacopla al observador de la fuente y le da control total sobre la suficiencia.



Etapa	Qué hace y qué garantía de SOT aporta
Source Connector	Trae bytes crudos de la fuente. El KQL es una "bomba de extracción", nunca lógica de hipótesis. Registra cobertura por-fuente (si una tabla falla, se marca ciega).
Raw Event Journal	Log append-only de cada evento tal como llegó, con rotación por tamaño para no llenar el disco. Permite *reconstruir* el grafo desde cero (base de confianza).
Normalizer	Convierte cada evento en un *sobre canónico*: id de fuente, tiempo de evento *y de ingesta*, procedencia, hash, payload crudo. Sin tipos de Azure en el núcleo.
GraphDelta	Contrato de cambios *idempotente*: re-ingestar el mismo evento no duplica aristas. Clave: dedup por *huella canónica*, nunca por timestamp.
Incremental Runtime	Aplica los deltas al grafo en memoria. Garantía verificada: *incremental* = *rebuild* (aplicar deltas da el mismo grafo que reconstruir desde el journal).
Domain API	Expone `run_hunt` y demás operaciones, agnóstico al transporte. GUI, REST y MCP invocan *el mismo* código local.
MCP / GUI	El consumidor. Recibe los caminos encontrados **y** los metadatos de cobertura, para nunca confundir "ciego" con "vacío".

4. Paso a paso: la vida de un evento

Sigamos un único evento — un login de Windows — desde que ocurre hasta que un analista caza sobre él. Este es el recorrido que hace real a SOT.

Paso 0. **Ocurre el hecho.** A las 14:30, la usuaria `alice` se autentica en `WORKSTATION-01`. Microsoft Sentinel lo registra como un `SecurityEvent` con `EventID 4624`. Este es el estado del mundo real, X.

Paso 1. **Extracción cruda.** El Source Connector polea Sentinel con una query *ligera*, sin joins ni agregaciones:

```
SecurityEvent
| extend _ingest_ts = coalesce(ingestion_time(), TimeGenerated)
| where _ingest_ts > datetime(<ultimo watermark>)
| order by _ingest_ts asc | take 10000
```

Por qué `ingestion_time()` y no `TimeGenerated`

El tiempo del *evento* (`TimeGenerated`) puede llegar "hacia atrás": un log tardío entra al workspace con un timestamp anterior al cursor y se perdería para siempre. El tiempo de *ingesta* solo crece, así que el cursor `> watermark` nunca se saltea una llegada tardía.

Paso 2. **Sobre canónico + journal.** El evento se envuelve en un `CanonicalEnvelope` que congela: el tiempo de evento y el de ingesta, la procedencia (qué mapping lo produjo), y un hash canónico determinístico. Se escribe al journal append-only.

Paso 3. **GraphDelta idempotente.** El normalizador emite un delta con clave `event_id` = `canonical_hash` = `sha256(payload ‖ mapping_hash)`. Si el conector se cae y re-polea la misma fila, el delta tiene la *misma* clave ⇒ el runtime lo reconoce como ya aplicado y **no duplica la arista**.

Paso 4. **Aplicación al grafo.** El runtime inserta la arista `User(alice) -[Auth]-> Host(WORKSTATION-01)`. La arista lleva: su tiempo de evento (para ordenar), su tiempo de *ingesta* (`ingested_at`, para razonar sobre llegada tardía), y su `mapping_hash` *congelado* (para procedencia estable).

Paso 5. **Fusión multi-fuente.** Otro evento — una conexión de FortiGate desde `WORKSTATION-01` a `10.0.0.42` — entra por el mismo camino. Como comparte el nodo `Host`, las dos fuentes se **fusionan en un solo grafo conectado**. Sentinel es solo una fuente más.

Paso 6. **La caza, 100 % local.** El analista lanza la hipótesis `User -[Auth]-> Host -[Connect]-> IP`. El `run_hunt` *traversa el grafo local* — sin volver a la nube, sin KQL — y encuentra el camino `alice` → `WORKSTATION-01` → `10.0.0.42`.

Paso 7. **La respuesta honesta.** La respuesta no es solo "1 camino". Incluye un objeto `AnswerCoverage`:

```
AnswerCoverage {
  possibly_incomplete: false,
  coverage_start: 1705328880, coverage_end: 1705329060,
  missing_sources: []
}
```

Si en cambio el conector de FortiGate hubiera fallado en esa ventana, la *misma* caza devolvería el mismo camino **pero** con `missing_sources: ["fortigate:edge-fw"]` y `possibly_incomplete: true`. *Ahí* está la diferencia entre "no lo vi" y "no pasó".

5. Los canales: en qué dimensiones puede fallar el observador

SOT descompone la suficiencia en **canales de telemetría**: dimensiones independientes en las que la observación puede ser fiel o ciega. Cada hipótesis de caza declara qué canales necesita. GraphHunter tiene 11 canales; siete son requeridos por las hipótesis del catálogo, y todos están hoy en **full**.

Canal	Pregunta que responde
`entity_identity`	¿Dos menciones de "alice" son la misma entidad?
`edge_multiplicity`	¿Cuántas veces ocurrió realmente esta relación? (dedup sin perder multiplicidad)
`event_time`	¿Cuándo ocurrió el evento?
`ingestion_time`	¿Cuándo lo *recibimos*? (para razonar sobre llegada tardía)
`field_survival`	¿Sobrevive la fila cruda para reconstruir campos no mapeados?
`row_completeness`	¿Registramos qué filas se descartaron por malformadas?
`ingest_completeness`	¿Sabemos si la ingesta se cortó a mitad (memoria crítica)?
`source_coverage`	¿Sabemos si una fuente quedó ciega en la ventana?
`skip_diagnostics`	¿Guardamos muestras de lo que se saltó?
`completeness_flag`	¿La respuesta puede decir "posiblemente incompleta"?
`provenance_stability`	¿Editar un mapping después de ingestar contamina la procedencia vieja?

Preguntas transversales

Además de las 30 hipótesis del catálogo, hay preguntas *transversales* que combinan canales. La más exigente es `loss_reconstruction` ("¿podemos reconstruir qué se perdió y por qué?"), que requiere `completeness_flag`, `ingestion_time`, `provenance_stability` y `skip_diagnostics`. Con los cuatro en **full**, esta transversal es **Suficiente**.

6. La auditoría: cómo se *mide* la suficiencia

La suficiencia no se afirma, se **audita**. Un runner (que vive en un repositorio separado, para preservar la cultura de resultados congelados) evalúa cada hipótesis contra el modelo de canales declarado y emite un veredicto:

Sufficient
conjunto ciego *vacío*: la respuesta es confiable

InsufficientWithWitness
hay un canal ciego ⇒ se adjunta el testigo concreto

El veredicto es puramente mecánico: una hipótesis es *Suficiente* si y solo si su *conjunto ciego* — los canales requeridos que no están en `full` — es vacío. Esto hace que cerrar un canal a `full` **propague** suficiencia de forma verificable y honesta.

Disciplina de claims (tabla de honestidad)

Cada afirmación de garantía en la documentación lleva una etiqueta: *proved / tested / implemented / read-verified / proposed*. Un canal se declara `full` solo con evidencia `file:line` concreta *y* con sus límites declarados. Ejemplos de límites que GraphHunter declara sin maquillar: el reloj de ingesta es el de *recepción* de GraphHunter (no el del origen); la identidad de un parser builtin es *code-versioned*; la ventana del journal es reciente-only.

7. Dos cierres de canal, en detalle

Para hacer tangible cómo se cierra un testigo, veamos dos casos reales.

7.1 `ingestion_time`: de "ciego" a "full"

El testigo: la arista guardaba `ingested_at_delta = 0` hardcodeado, así que un evento tardío y uno puntual producían la *misma* arista — `same_observed`.

El cierre: (1) los sitios de ingesta ahora estampan `encode_ingested_at(now())`; (2) el valor se *preserva* en la compactación; (3) se *lee de vuelta* en la arista materializada. Ahora un evento tardío estampa un `ingested_at` estrictamente posterior ⇒ el testigo se rompe. Verificado por un test que ingesta un evento y verifica que su `ingested_at` ≈ ahora, no la época base.

7.2 `provenance_stability`: el testigo que casi se nos escapa

El testigo: editar un mapping después de ingestar no debe cambiar la procedencia de aristas viejas. Un análisis inicial concluyó "ya está cerrado" — pero la revisión adversarial encontró un camino de contaminación real: una función que *sobreescribía* el `field_config` de un dataset ya ingestado, y el export leía ese valor *mutable*.

El cierre: se congeló la identidad del mapping (`DatasetInfo.mapping_hash`) *separada* del `field_config` mutable; el export lee solo el valor congelado. Un test *genuino* ingesta bajo el config A, edita a config B, re-exporta, y verifica que las aristas viejas mantienen el hash de A.

La lección

Este caso es el espíritu de SOT en acción: la disciplina de *verificar contra el código, no contra la nota*, y de dejar que una revisión adversarial cace lo que el autor pasó por alto. La honestidad no es un adorno; es el mecanismo.

8. El certificado incremental = rebuild

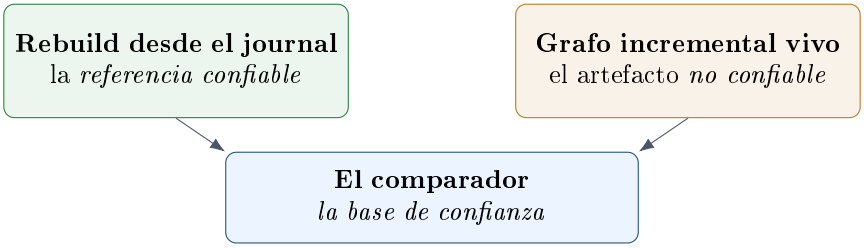
Hay una grieta silenciosa en toda esta historia. GraphHunter no reconstruye el grafo desde cero en cada ingesta — eso sería carísimo. Aplica *deltas* incrementales a un grafo que ya vive en memoria. Pero entonces surge una duda que va al corazón de la confianza:

La pregunta de fondo

El camino *rápido* (aplicar deltas incrementalmente) ¿produce **el mismo grafo** que el camino *lento y confiable* (reconstruir todo desde el journal)? Si no lo hiciera, toda garantía de cobertura o suficiencia sobre el grafo vivo sería una ilusión: estaríamos razonando sobre un artefacto que no es fiel a la evidencia.

8.1 Validación por traducción: quién es la base de confianza

GraphHunter adopta aquí un patrón tomado de la verificación de compiladores (SOT-VM): la **validación por traducción**. En vez de *confiar* en que el runtime incremental es correcto, se lo trata como un "compilador no confiable" y se **verifica su salida** contra una referencia.



La base de confianza no es el runtime incremental (que puede tener bugs), sino el **comparador** más la referencia. Si el comparador dice "iguales", entonces — para todo lo que el comparador sabe mirar — el camino rápido es tan fiel como el lento.

8.2 Qué compara el comparador (y qué honestamente *no*)

El comparador toma una foto de ambos grafos (`to_snapshot()`, resolviendo las identidades internas a cadenas de texto para poder compararlas entre grafos) y contrasta dimensión por dimensión:

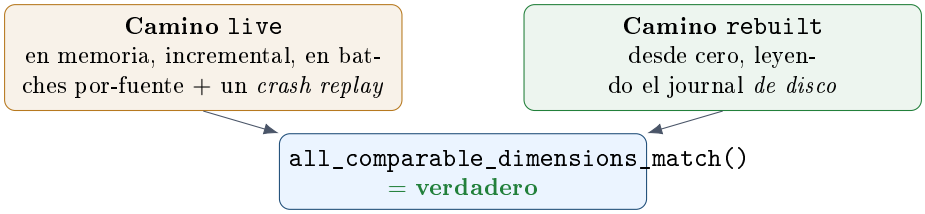
Dimensión	Cómo se compara
`entities`	Conjunto de ids de entidades.
`entity_metadata`	Por id: tipo (por *valor*, no por texto), metadata y `dataset_id`.
`edges_multiset`	*Multiset* de aristas `(src, rel, dst, ts, meta, dataset)`: la multiplicidad se atrapa por construcción; el orden no se asume.
`temporal_window`	El rango `[min, max]` de timestamps de aristas.
`hypothesis_results`	Una batería de hipótesis corrida en *ambos* grafos; se comparan los caminos como multiset.

La honestidad del `NotComparable`

Hay dimensiones que el grafo **no observa**: la procedencia más allá del `dataset_id`, los scores de anomalía (los produce un pase aparte), el timestamp MVCC de inserción (no determinístico entre live y rebuild). El comparador las marca `NotComparable` *con su razón* — nunca como "iguales". Afirmar igualdad sobre una dimensión que no se puede observar es justamente la sobre-afirmación que SOT prohíbe. Del mismo modo, si la batería de hipótesis no produce ningún camino en ninguno de los dos grafos, el resultado es `NotComparable` ("silencio ≠ acuerdo"), no un `Match` vacío.

8.3 El certificado sobre un dataset *real*

La prueba definitiva no corre sobre datos sintéticos, sino sobre un dataset de intrusión **multi-fuente real** (Sysmon + Sentinel + FortiGate), ingerido por *dos caminos genuinamente distintos*:



Es una equivalencia real *memoria-vs-disco* + *incremental-vs-batch* + *dedup* (el crash replay reaplica un batch y el conteo **no** crece). Resultado: 9 entidades / 7 relaciones, las cinco dimensiones comparables en `PASS`, y la procedencia/scores honestamente `NotComparable`.

El control negativo: el certificado tiene dientes

Un certificado que siempre dice "iguales" no prueba nada. Por eso hay un *control negativo*: se dropea el último registro del journal, de modo que la reconstrucción sea un subconjunto estricto del grafo vivo. El comparador entonces marca `DIFF` correctamente. Un certificado que pasa *y* falla cuando debe fallar es un certificado en el que se puede confiar.

8.4 Un límite declarado: `rebuild_faithful`

La reconstrucción todavía no reproduce fielmente los sobres de `Correction`/`Tombstone` (correcciones y borrados lógicos). Si el journal validado contiene alguno, el reporte marca `rebuild_faithful: false` con un caveat — la equivalencia **no** se afirma sobre un journal con correcciones. Otro ejemplo de la disciplina: declarar el límite en vez de maquillarlo.

Por qué esto cierra el círculo

El certificado incremental = rebuild es lo que hace *confiables* a todas las garantías anteriores. La cobertura, la multiplicidad, la procedencia congelada — todas se calculan sobre el grafo vivo (el camino rápido). El certificado prueba que ese grafo rápido es fiel a la evidencia del journal (el camino confiable). Sin él, SOT razonaría sobre una sombra; con él, razona sobre el grafo real.

9. El estado final: todo Suficiente, con límites honestos

Tras cerrar los canales, el estado de la auditoría es:

Familia	Veredicto
30 hipótesis del catálogo ATT&CK	**Sufficient** (30/30)
Transversal `exhaustiveness_marking`	**Sufficient**
Transversal `loss_reconstruction`	**Sufficient**
Los 11 canales de telemetría	**full**

Y la maquinaria corre **por defecto** (journal acotado + dedup idempotente + cobertura + cobertura por-fuente), con cada flag como un escape hatch que se puede apagar. Lo que *no* se afirma: que el repositorio sea "Suficiente en todo" de forma incondicional. Cada garantía arrastra su límite declarado — y eso, precisamente, es lo que la hace confiable.

La idea para llevarse

SOT no le pide al observador que sea omnisciente. Le pide que sea **honesto sobre su ceguera**. En GraphHunter, eso se traduce en una regla simple: *una respuesta de caza nunca presenta datos incompletos como exhaustivos*. Cuando el sistema no puede ver algo, lo dice — y esa distinción, entre "no pasó" y "no lo vi", es lo que separa a un cazador de amenazas confiable de uno que solo parece confiable.

Documento generado a partir de la arquitectura de GraphHunter y su serie de validación SOT.